CO3 ASSIGNMENT – 3

Complete Solutions, C Programs and Complexity Analysis

Note: The solutions below follow the questions and expected behaviour in the supplied assignment. Programs are independent C examples; compile each program separately. For large experimental measurements, run the programs with the requested input sizes and record the values on your computer.

Q1 – Searching a Library Catalogue

1. Binary search: completed C program

```
#include <stdio.h>

int steps;

int binary_search(const int a[], int n, int key)
{
    int low = 0, high = n - 1;
    steps = 0;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        steps++;                        /* one key comparison at a[mid] */

        if (a[mid] == key)
            return mid;
        else if (a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main(void)
{
    int accession[] = {1021, 1048, 1103, 1190, 1222,
                       1305, 1377, 1402, 1460, 1533};
    int n = sizeof accession / sizeof accession[0];
    int keys[] = {1021, 1305, 1533, 1400};

    for (int k = 0; k < 4; k++) {
        int pos = binary_search(accession, n, keys[k]);
        printf("Book %d -> index %d, comparisons = %d\\n",
               keys[k], pos, steps);
    }
    return 0;
}
```

Output

```
Book 1021 -> index 0, comparisons = 3
Book 1305 -> index 5, comparisons = 3
Book 1533 -> index 9, comparisons = 4
Book 1400 -> index -1, comparisons = 4
```

2. Linear-search pseudocode

```
LINEAR_SEARCH(A, n, key)
    for i = 0 to n - 1
        if A[i] == key
            return i
    return -1
```

3. Binary-search pseudocode

```
BINARY_SEARCH(A, n, key)   // A must be sorted
    low = 0; high = n - 1
    while low <= high
        mid = low + (high - low) / 2
        if A[mid] == key return mid
        if A[mid] < key then low = mid + 1
        else high = mid - 1
    return -1
```

4. Comparison counts and complexity

Linear search: Best = 1 comparison; worst = n comparisons; average successful search = (n + 1) / 2 comparisons when each position is equally likely. An unsuccessful search uses n comparisons. Time is O(n), extra space O(1).

Binary search: Best = 1 comparison (key is at the middle); worst is approximately floor(log■ n) + 1 key comparisons; average is O(log n). Time is O(log n), extra space O(1) for the iterative version.

For n = 100,000, linear search needs at most 100,000 comparisons and about 50,000.5 comparisons on average for a successful search. Binary search needs at most about 17 comparisons (since 2^{1}■ < 100,000 < 2^{1}■).

Why sorted data? Binary search discards half the remaining range after each comparison. This is valid only when the values are in order, because the comparison tells us which half can contain the key.

Why use mid = low + (high – **low) / 2?** It avoids the possible integer overflow of (low + high) / 2 when low and high are very large.

Why can a missing key need worst-case comparisons? The algorithm must keep halving the possible range until it becomes empty. A missing key may therefore require all levels of the search, just like a key found at the deepest level.

5. Linear search versus binary search; daily additions

Keeping the array sorted is worthwhile when searches are frequent and insertions are relatively rare. Binary search makes each lookup O(log n), but inserting into an array may require shifting O(n) elements to preserve order. The decision needs the number of searches and insertions per day, array size, cost of shifting, whether data arrives in batches, and whether another structure (such as a balanced tree or hash table) is allowed.

6. Alternative approaches

Hash table with linear probing: Average search is O(1) when the hash table has a good hash function and a controlled load factor; worst case is O(n). Extra space is O(table size), usually O(n).

Interpolation search: On uniformly distributed sorted numeric keys, average time can be O(log log n); worst case is O(n). It uses O(1) extra space. It still requires sorted data.

Q2 – Sorting the Examination Hall Seating Register

1. C implementation of bubble, selection and insertion sort

```
#include <stdio.h>

void bubble_sort(int a[], int n, long *comp, long *swaps)
{
    *comp = *swaps = 0;
    for (int i = 0; i < n - 1; i++) {
        int changed = 0;
        for (int j = 0; j < n - 1 - i; j++) {
            (*comp)++;
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                (*swaps)++;
                changed = 1;
            }
        }
        if (!changed) break;
    }
}

void selection_sort(int a[], int n, long *comp, long *swaps)
{
    *comp = *swaps = 0;
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++) {
            (*comp)++;
            if (a[j] < a[min]) min = j;
        }
        if (min != i) {
            int t = a[i]; a[i] = a[min]; a[min] = t;
            (*swaps)++;
        }
    }
}

void insertion_sort(int a[], int n, long *comp, long *shifts)
{
    *comp = *shifts = 0;
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0) {
            (*comp)++;
            if (a[j] > key) {
                a[j + 1] = a[j];
                (*shifts)++;
```

```
                j--;
            } else break;
        }
        a[j + 1] = key;
    }
}

int main(void)
{
    int x[] = {42, 18, 35, 11, 29, 7};
    int n = sizeof x / sizeof x[0];
    long c, s;
    insertion_sort(x, n, &c, &s);
    printf("Insertion sort: comparisons=%ld shifts=%ld\\n", c, s);
    for (int i = 0; i < n; i++) printf("%d ", x[i]);
    printf("\\n");
    return 0;
}
```

2. Trace of insertion sort on six values

```
Start: 42 18 35 11 29 7
Insert 18: 18 42 35 11 29 7
Insert 35: 18 35 42 11 29 7
Insert 11: 11 18 35 42 29 7
Insert 29: 11 18 29 35 42 7
Insert 7 : 7 11 18 29 35 42
```

3. Complexity and behaviour

Bubble sort with early exit: best O(n) on already sorted input; average and worst O(n²). The flag stops the algorithm when a full pass makes no swaps. Selection sort: O(n²) comparisons in best, average and worst cases, with at most n–1 swaps. Insertion sort: best O(n) on sorted input; average and worst O(n²).

Insertion sort is fast on nearly sorted data because each item moves only a short distance and few shifts are needed. It is stable when it shifts only values strictly greater than the key. Bubble sort is stable when equal elements are not swapped. Standard selection sort is not stable because a distant minimum-value swap can change the order of equal elements.

Stability matters when equal-mark students must retain a previous ordering, such as alphabetical order, after sorting by marks.

4. Expected comparison counts

For n = 6 in the trace above, insertion sort produces 12 key comparisons and 11 shifts (the final failed comparison is counted for each insertion where it occurs). Exact counts depend on the input order and the counting convention.

For n = 100, 1,000 and 10,000, record measured comparisons and swaps/shifts separately for sorted, nearly sorted, reverse-sorted and random arrays. Do not invent measured timings: they depend on the machine and test data.

5. Recommendation for a mostly sorted register

Use insertion sort when the register is already sorted except for about 20 late registrations. It is simple and performs close to O(n + k) work when only a small number of items are out of place; bubble sort with early exit may also perform well, but insertion sort generally handles this situation more directly.

6. Counting sort alternative

If register numbers are integers in a known, reasonably small range k, counting sort counts occurrences and writes the values back in order. Time is O(n + k) and extra space is O(k). It is unsuitable when k is extremely large compared with n, when keys are not discrete integers, or when the original relative order of records must be preserved without additional handling.

Q3 – Merging Two Merit Lists

1. Completed C program

```
#include <stdio.h>

int merge(const int a[], int na, const int b[], int nb, int out[])
{
    int i = 0, j = 0, k = 0;

    while (i < na && j < nb) {
        if (a[i] >= b[j]) out[k++] = a[i++];
        else              out[k++] = b[j++];
    }
    while (i < na) out[k++] = a[i++];
    while (j < nb) out[k++] = b[j++];
```

```
    return k;
}

int main(void)
{
    int campusA[] = {98, 91, 85, 72};
    int campusB[] = {95, 91, 80};
    int merged[7];
    int n = merge(campusA, 4, campusB, 3, merged);

    for (int i = 0; i < n; i++) printf("%d ", merged[i]);
    printf("\\n");
    return 0;
}
```

Output

```
98 95 91 91 85 80 72
```

2. Pseudocode

```
MERGE(A, m, B, n)
    i = 0; j = 0; k = 0
    while i < m and j < n
        if A[i] >= B[j]
            out[k] = A[i]; i = i + 1
        else
            out[k] = B[j]; j = j + 1
        k = k + 1
    copy remaining A elements to out
    copy remaining B elements to out
    return out
```

3. Analysis

Each iteration compares the current unmerged element of each list and copies one element. Once one list ends, the other list is already sorted, so its remaining elements can be copied directly. Each element is copied once. Time is O(m + n); the output array uses O(m + n) extra space.

The number of score comparisons is at most m + n – 1 when both lists are non-empty; it may be smaller depending on which list ends first. If one list is empty, comparisons = 0 and the other list is copied.

The code uses >=, so equal scores from campus A are placed before equal scores from campus B. This preserves the relative order within each campus and gives campus A priority for ties. Reversing the condition gives campus B priority.

4. Merge versus concatenate and sort

Joining the arrays and sorting again takes O((m+n) log(m+n)) time with an efficient comparison sort. Merging takes O(m+n), which is better when both lists are already sorted. For m=n=1,000, merge is linear in 2,000 elements, while sorting again is proportional to 2,000 log■2,000. For m=n=10,000, merge is linear in 20,000, while sorting again is proportional to 20,000 log■20,000.

An in-place merge avoids a separate output array but is more complicated. A simple shifting-based in-place method can take O(m·n) time and O(1) auxiliary space. It may be preferred when memory is severely limited and the slower processing time is acceptable.

Q4 – Two-Dimensional Arrays: District Rainfall Records

1. C functions for the four computations

```
#include <stdio.h>

#define DISTRICTS 38
#define MONTHS 12

void row_totals(const double rain[DISTRICTS][MONTHS],
                double totals[DISTRICTS])
{
    for (int i = 0; i < DISTRICTS; i++) {
        totals[i] = 0;
        for (int j = 0; j < MONTHS; j++)
            totals[i] += rain[i][j];
    }
}

void monthly_averages(const double rain[DISTRICTS][MONTHS],
                      double avg[MONTHS])
{
    for (int j = 0; j < MONTHS; j++) {
        double sum = 0;
```

```
        for (int i = 0; i < DISTRICTS; i++)
            sum += rain[i][j];
        avg[j] = sum / DISTRICTS;
    }
}

void wettest(const double rain[DISTRICTS][MONTHS],
             int *district, int *month, double *value)
{
    *value = rain[0][0]; *district = 0; *month = 0;
    for (int i = 0; i < DISTRICTS; i++)
        for (int j = 0; j < MONTHS; j++)
            if (rain[i][j] > *value) {
                *value = rain[i][j];
                *district = i; *month = j;
            }
}

void deficient(const double rain[DISTRICTS][MONTHS],
               const double totals[DISTRICTS], double state_avg)
{
    for (int i = 0; i < DISTRICTS; i++)
        if (totals[i] < 0.60 * state_avg)
            printf("District %d is below 60%% of state average\\n", i + 1);
}

int main(void)
{
    static double rain[DISTRICTS][MONTHS] = {{0}};
    double totals[DISTRICTS], avg[MONTHS], total = 0, state_avg;
    int d, m; double max_value;

    /* Replace the zero initialization with actual rainfall input. */
    row_totals(rain, totals);
    monthly_averages(rain, avg);
    wettest(rain, &d, &m, &max_value);

    for (int i = 0; i < DISTRICTS; i++) total += totals[i];
    state_avg = total / DISTRICTS; /* average annual rainfall per district */

    printf("Wettest: district %d, month %d, %.2f mm\\n",
           d + 1, m + 1, max_value);
    deficient(rain, totals, state_avg);
    return 0;
}
```

2. Address formula and memory layout

C stores a two-dimensional array in row-major order: all 12 months of district 0, then all 12 months of district 1, and so on. If the base address is B and each element occupies w bytes, the address of rain[i][j] is B + ((i × 12) + j) × w.

Row-wise traversal is usually faster because adjacent elements are contiguous in memory and make good use of cache lines. Column-wise traversal jumps between rows and can cause more cache misses for large arrays.

A three-dimensional extension is double rain[years][38][12], where each year contains 38 district rows and each row contains 12 months.

3. Hand verification using a 3 × 4 example

```
Array:
1  2  3  4
5  6  7  8
9 10 11 12

Row totals: 10, 26, 42
Column averages: 5, 6, 7, 8
Maximum: 12 at row 3, column 4
```

4. Why use a 2-D array?

A 2-D array groups all district-month values in one clearly indexed structure. It simplifies loops, function parameters and calculations, and stores the values contiguously. Thirty-eight separate arrays would be harder to pass, maintain and traverse consistently.

A flat one-dimensional array can use index i×12+j and has O(1) access with contiguous storage. An array of 38 pointers also provides O(1) indexed access, but needs space for 38 pointers and may allocate rows separately, causing less predictable locality and extra allocation overhead. All three approaches have O(1) element access; layout affects practical speed.

Q5 – Processing Text Without the Library

1. Completed C program without string.h

```
#include <stdio.h>
#include <ctype.h>

int word_count(const char s[])
{
    int count = 0, inside = 0;
    for (int i = 0; s[i] != '\\0'; i++) {
        if (s[i] != ' ' && s[i] != '\\t' &&
            s[i] != '\\n' && s[i] != '\\r') {
            if (!inside) count++;
            inside = 1;
        } else {
            inside = 0;
        }
    }
    return count;
}

int is_palindrome(const char s[])
{
    int left = 0, right = 0;
    while (s[right] != '\\0') right++;
    right--;

    while (left < right) {
        while (left < right && !isalpha((unsigned char)s[left]))
            left++;
        while (left < right && !isalpha((unsigned char)s[right]))
            right--;

        if (tolower((unsigned char)s[left]) !=
            tolower((unsigned char)s[right]))
            return 0;
        left++; right--;
    }
    return 1;
}

int main(void)
{
    printf("Words: %d\\n", word_count(" Meet at the library at 5 PM "));
    printf("%d %d\\n",
           is_palindrome("Madam, I'm Adam"), is_palindrome("Hello"));
    printf("Tab test: %d\\n", word_count("one\\ttwo three"));
    return 0;
}
```

Output

```
Words: 7
1 0
Tab test: 3
```

The assignment's printed expected output shows “10” on one line, but the two printf arguments produce the values 1 and 0 separated by a space: the first phrase is a palindrome when punctuation and case are ignored; “Hello” is not.

2. Pseudocode for word counting

```
count = 0; inside = false
for each character until '\0'
    if character is not whitespace
        if inside is false: count++
        inside = true
    else
        inside = false
return count
```

3. String concepts and safety

The null character '\0' marks the end of a C string. Without it, string functions may continue reading beyond the intended text. char s[] = "OK" creates a modifiable character array containing O, K and '\0'. char *s = "OK" points to a string literal, which must not be modified.

Repeatedly calculating strlen(s) in a loop condition makes the loop $O(n^2)$, because strlen scans the string each time. Calculate the length once or stop at '\0'.

Copying a long string into a smaller destination with strcpy causes a buffer overflow and undefined behaviour. Use a correctly sized destination and bounded input such as fgets; snprintf can be used for formatted output. gets() was removed because it cannot limit the number of characters read and therefore cannot prevent buffer overflow.

Use your own string functions to learn or when a specialized operation is needed. Use standard library functions for common, tested operations, while checking destination sizes and input limits.

4. Library-function alternatives

strlen returns string length, strcpy copies a string, strcmp compares strings lexicographically, and toupper converts a character to uppercase. Use for the first three and for toupper. For bounded copying, ensure the destination is terminated; snprintf is often easier to use safely.

Q6 – A Directory of Names: Sorting and Searching Strings

1. C program: read, sort, binary-search and identify duplicates

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 500
#define LEN 30

int main(void)
{
    char names[MAX][LEN], temp[LEN], key[LEN];
    int n;

    printf("Number of names: ");
    if (scanf("%d", &n) != 1 || n < 0 || n > MAX) return 1;
    getchar(); /* consume newline after number */

    for (int i = 0; i < n; i++) {
        if (fgets(names[i], LEN, stdin) == NULL) return 1;
        names[i][strcspn(names[i], "\\n")] = '\\0';
    }

    /* Bubble sort using strcmp; stable for equal strings. */
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (strcmp(names[j], names[j + 1]) > 0) {
                strcpy(temp, names[j]);
                strcpy(names[j], names[j + 1]);
                strcpy(names[j + 1], temp);
            }
        }
    }

    printf("Sorted names:\\n");
    for (int i = 0; i < n; i++) {
        puts(names[i]);
        if (i > 0 && strcmp(names[i], names[i - 1]) == 0)
            printf("Duplicate: %s\\n", names[i]);
    }

    printf("Search name: ");
    if (fgets(key, LEN, stdin) == NULL) return 1;
    key[strcspn(key, "\\n")] = '\\0';

    int low = 0, high = n - 1, found = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        int cmp = strcmp(names[mid], key);
        if (cmp == 0) { found = mid; break; }
        if (cmp < 0) low = mid + 1;
        else high = mid - 1;
    }

    if (found >= 0) printf("Found at index %d\\n", found);
    else puts("Name not found");
    return 0;
}
```

2. Explanation

strcmp compares character codes from left to right. It returns a negative value if the first string is lexicographically smaller, zero if equal, and a positive value if greater. Uppercase and lowercase letters have different character codes, so “Apple” and “apple” may be ordered differently.

A case-insensitive comparison can compare tolower((unsigned char)a[i]) and tolower((unsigned char)b[i]) character by character until a difference or the end of either string.

After sorting, identical names are adjacent, so duplicates can be detected by comparing each name with its previous neighbour. Swapping two names by copying characters costs O(L), where L is the maximum name length. Sorting indices

or pointers swaps small fixed-size values instead.

3. Length 29 and 30

The array char name[30] can store at most 29 visible characters plus the terminating '\0'. A 30-character input will not fit completely with its terminator; fgets reads at most 29 characters and leaves the remainder in the input stream. Increase the array size if 30 visible characters must be accepted.

4. Complexity and pointer alternative

Bubble sorting n names uses $O(n^2)$ string comparisons; each strcmp may inspect up to L characters, so a rough upper bound is $O(n^2L)$. Binary search uses O(log n) string comparisons, each up to O(L), giving O(L log n).

Sorting an array of pointers with qsort() avoids copying the full character arrays during swaps. Sorting still needs O(n log n) comparisons, each potentially O(L) character work; the pointer array uses O(n) extra storage. The trade-off is that the pointed-to strings must remain valid and their storage must be managed correctly.

Q7 – Design an Election Counting Program from First Principles

1. Inputs, outputs and rules

• Input: candidate count c (maximum 12), vote count v (maximum 5,000), then v candidate numbers.

• Valid vote: candidate number from 1 through c. Invalid vote: 0, negative, or greater than c.

• Output: valid and invalid totals, each candidate's votes in descending order, winner and tie status.

• Arrays: count[c+1] for candidate totals and order[c] for candidate indices.

2. C implementation

```
#include <stdio.h>

#define MAX_CANDIDATES 12
#define MAX_VOTES 5000

typedef struct {
    int id;
    int votes;
} Result;

int main(void)
{
    int c, v, vote;
    int count[MAX_CANDIDATES + 1] = {0};
    int valid = 0, invalid = 0;

    printf("Candidates and votes: ");
    if (scanf("%d %d", &c, &v) != 2 ||
        c < 1 || c > MAX_CANDIDATES ||
        v < 0 || v > MAX_VOTES) {
        puts("Invalid input");
        return 1;
    }

    for (int i = 0; i < v; i++) {
        if (scanf("%d", &vote) != 1) return 1;
        if (vote >= 1 && vote <= c) {
            count[vote]++;
            valid++;
        } else {
            invalid++;
        }
    }

    Result r[MAX_CANDIDATES];
    for (int i = 0; i < c; i++) {
        r[i].id = i + 1;
        r[i].votes = count[i + 1];
    }

    /* Selection sort by votes, descending. */
    for (int i = 0; i < c - 1; i++) {
        int best = i;
        for (int j = i + 1; j < c; j++)
            if (r[j].votes > r[best].votes) best = j;
        if (best != i) {
            Result t = r[i]; r[i] = r[best]; r[best] = t;
        }
    }
```

```
    printf("Valid votes: %d\\nInvalid votes: %d\\n", valid, invalid);
    puts("Candidate results (descending):");
    for (int i = 0; i < c; i++)
        printf("Candidate %d: %d\\n", r[i].id, r[i].votes);

    if (c > 0) {
        int tied = 0;
        for (int i = 1; i < c; i++)
            if (r[i].votes == r[0].votes) tied = 1;

        if (tied) {
            printf("Tie for highest votes: %d votes\\n", r[0].votes);
            for (int i = 0; i < c; i++)
                if (r[i].votes == r[0].votes)
                    printf("Candidate %d ", r[i].id);
            printf("\\n");
        } else {
            printf("Winner: Candidate %d with %d votes\\n",
                   r[0].id, r[0].votes);
        }
    }
    return 0;
}
```

3. Example test

```
Input:
3 8
1 2 2 3 0 4 1 2

Output:
Valid votes: 6
Invalid votes: 2
Candidate results (descending):
Candidate 2: 3
Candidate 1: 2
Candidate 3: 1
Winner: Candidate 2 with 3 votes
```

For a tie test, use input: 2 4 followed by 1 1 2 2. Both candidates receive 2 votes, so the program reports a tie.

4. Design defence and alternatives

A count array indexed by candidate number processes each vote in O(1), so counting v votes takes O(v) time and O(c) space. Searching a list of candidate names for every vote takes O(v·c) time in the simple linear-search approach. Sorting all votes and counting equal runs takes O(v log v) time and O(v) storage if a separate sorted copy is used.

With 500 candidates, increase the candidate-array capacity or allocate it dynamically; the counting approach remains O(v+c). One limitation is that candidate numbers must be valid, consistent identifiers. The program also assumes each entered vote is a single integer and does not provide audit/security checks for a real election.

5. Complexity summary

Count array: O(v+c) time including initialization/output, O(c) extra space. **Search candidate list for each vote:** O(v·c) time, O(c) space for the list. **Sort votes then count runs:** O(v log v) time, typically O(v) extra space for a copied array (sorting algorithm dependent).

Overall Complexity Summary

```
Q1 Linear search:          O(n) time, O(1) space
   Binary search:          O(log n) time, O(1) space
   Hash table:             O(1) average search, O(n) worst

Q2 Bubble sort:            O(n) best with early exit; O(n^2) worst
   Selection sort:         O(n^2) comparisons
   Insertion sort:         O(n) best; O(n^2) worst
   Counting sort:          O(n+k) time, O(k) space

Q3 Merge two sorted lists: O(m+n) time, O(m+n) output space
   Concatenate + sort:     O((m+n) log(m+n)) time

Q4 2-D array element access: O(1)
   Row-wise traversal:     better locality in row-major C arrays

Q5 Normal string loops:    O(n) time
   strlen in loop test:    O(n^2) time
   Recursive reversal:    O(n) time, O(n) call-stack space

Q6 Name sort:              O(n^2 L) for bubble sort
   Binary name search:    O(L log n)
   qsort pointer sort:    O(n log n) comparisons, O(n) pointer storage
```

```
Q7 Count array:            O(v+c) time, O(c) space
   Search per vote:       O(v*c) time
   Sort votes:            O(v log v) time
```

End of solutions. Replace sample data with the values specified by your lecturer when performing the experimental parts.